# Using Dataclasses

FastAPI is built on top of **Pydantic**, and I have been showing you how to use Pydantic models to declare requests and responses.

But FastAPI also supports using `dataclasses` the same way:

Python 3.10+

```python
from dataclasses import dataclass

from fastapi import FastAPI


@dataclass
class Item:
    name: str
    price: float
    description: str | None = None
    tax: float | None = None


app = FastAPI()


@app.post("/items/")
async def create_item(item: Item):
    return item
```

This is still supported thanks to **Pydantic**, as it has internal support for `dataclasses`.

So, even with the code above that doesn't use Pydantic explicitly, FastAPI is using Pydantic to convert those standard dataclasses to Pydantic's own flavor of dataclasses.

And of course, it supports the same:

- data validation
- data serialization
- data documentation, etc.

This works the same way as with Pydantic models. And it is actually achieved in the same way underneath, using Pydantic.

> **Info**
>
> Keep in mind that dataclasses can't do everything Pydantic models can do.
>
> So, you might still need to use Pydantic models.
>
> But if you have a bunch of dataclasses laying around, this is a nice trick to use them to power a web API using FastAPI. 🤓

## Dataclasses in `response_model`

Ask AI

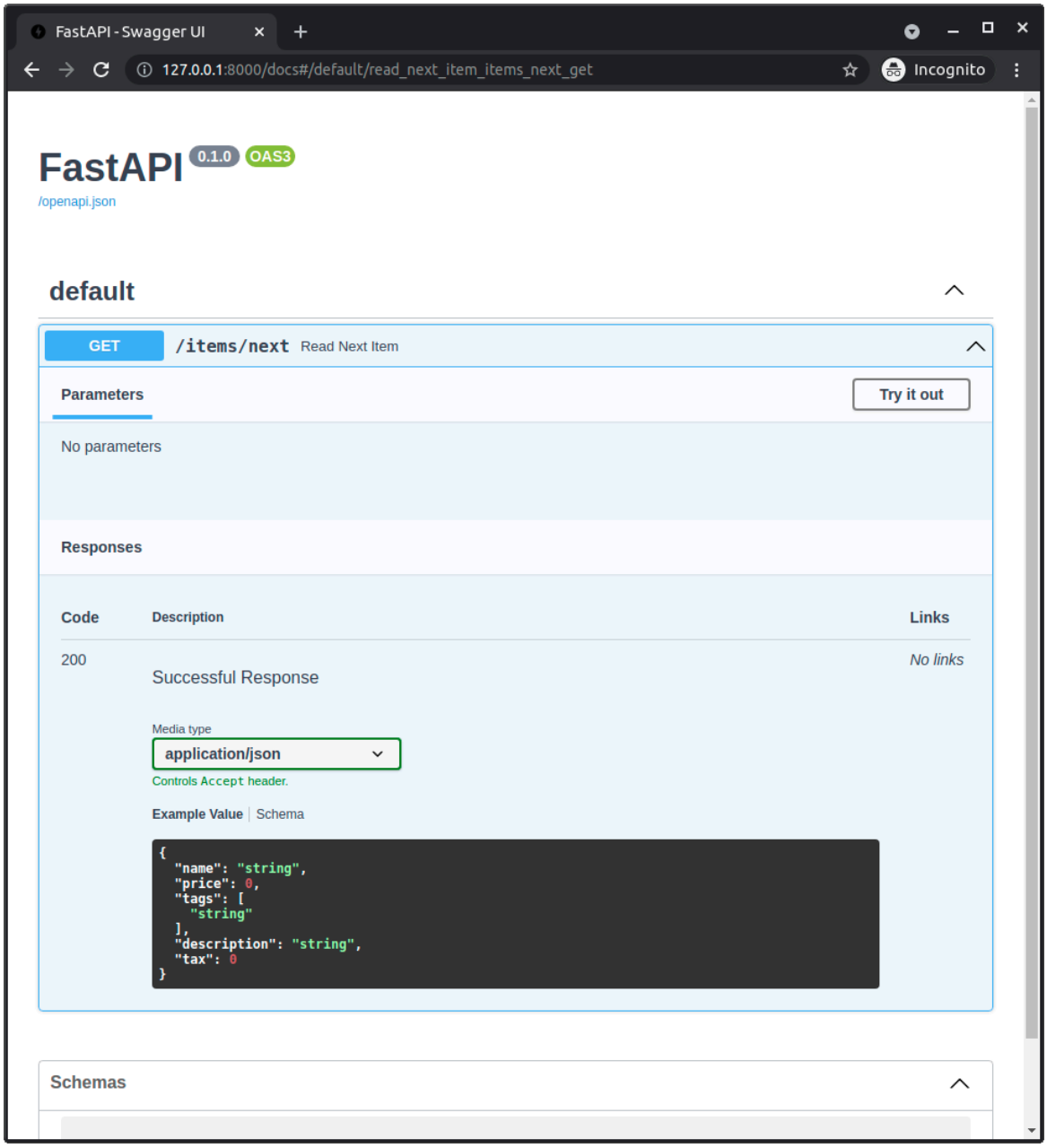
You can also use `dataclasses` in the `response_model` parameter:

Python 3.10+

```python
from dataclasses import dataclass, field

from fastapi import FastAPI


@dataclass
class Item:
    name: str
    price: float
    tags: list[str] = field(default_factory=list)
    description: str | None = None
    tax: float | None = None


app = FastAPI()


@app.get("/items/next", response_model=Item)
async def read_next_item():
    return {
        "name": "Island In The Moon",
        "price": 12.99,
        "description": "A place to be playin' and havin' fun",
        "tags": ["breater"],
    }
```

The dataclass will be automatically converted to a Pydantic dataclass.

This way, its schema will show up in the API docs user interface:




## Dataclasses in Nested Data Structures

You can also combine `dataclasses` with other type annotations to make nested data structures.

In some cases, you might still have to use Pydantic's version of `dataclasses`. For example, if you have errors with the automatically generated API documentation.

In that case, you can simply swap the standard `dataclasses` with `pydantic.dataclasses`, which is a drop-in replacement:

Python 3.10+

```
from dataclasses import field  # (1)

from fastapi import FastAPI
from pydantic.dataclasses import dataclass  # (2)
```

```python
@dataclass
class Item:
    name: str
    description: str | None = None


@dataclass
class Author:
    name: str
    items: list[Item] = field(default_factory=list)  # (3)


app = FastAPI()


@app.post("/authors/{author_id}/items/", response_model=Author)  # (4)
async def create_author_items(author_id: str, items: list[Item]):  # (5)
    return {"name": author_id, "items": items}  # (6)


@app.get("/authors/", response_model=list[Author])  # (7)
def get_authors():  # (8)
    return [  # (9)
        {
            "name": "Breaters",
            "items": [
                {
                    "name": "Island In The Moon",
                    "description": "A place to be playin' and havin' fun",
                },
                {"name": "Holy Buddies"},
            ],
        },
        {
            "name": "System of an Up",
            "items": [
                {
                    "name": "Salt",
                    "description": "The kombucha mushroom people's favorite",
                },
                {"name": "Pad Thai"},
                {
                    "name": "Lonely Night",
                    "description": "The mostests lonliest nightiest of allest",
                },
            ],
        },
    ]
```

1. We still import `field` from standard `dataclasses`.
2. `pydantic.dataclasses` is a drop-in replacement for `dataclasses`.
3. The `Author` dataclass includes a list of `Item` dataclasses.
4. The `Author` dataclass is used as the `response_model` parameter.
5. You can use other standard type annotations with dataclasses as the request body.

   In this case, it's a list of `Item` dataclasses.
6. Here we are returning a dictionary that contains `items` which is a list of dataclasses.

   FastAPI is still capable of *serializing* the data to JSON.
7. Here the `response_model` is using a type annotation of a list of `Author` dataclasses.

   Again, you can combine `dataclasses` with standard type annotations.
8. Notice that this *path operation function* uses regular `def` instead of `async def`.

   As always, in FastAPI you can combine `def` and `async def` as needed.

If you need a refresher about when to use which, check out the section *"In a hurry?"* in the docs about `async` and `await`.

9. This *path operation function* is not returning dataclasses (although it could), but a list of dictionaries with internal data.

   FastAPI will use the `response_model` parameter (that includes dataclasses) to convert the response.

You can combine `dataclasses` with other type annotations in many different combinations to form complex data structures.

Check the in-code annotation tips above to see more specific details.

## Learn More

You can also combine `dataclasses` with other Pydantic models, inherit from them, include them in your own models, etc.

To learn more, check the Pydantic docs about dataclasses.

## Version

This is available since FastAPI version `0.67.0`. 🔖